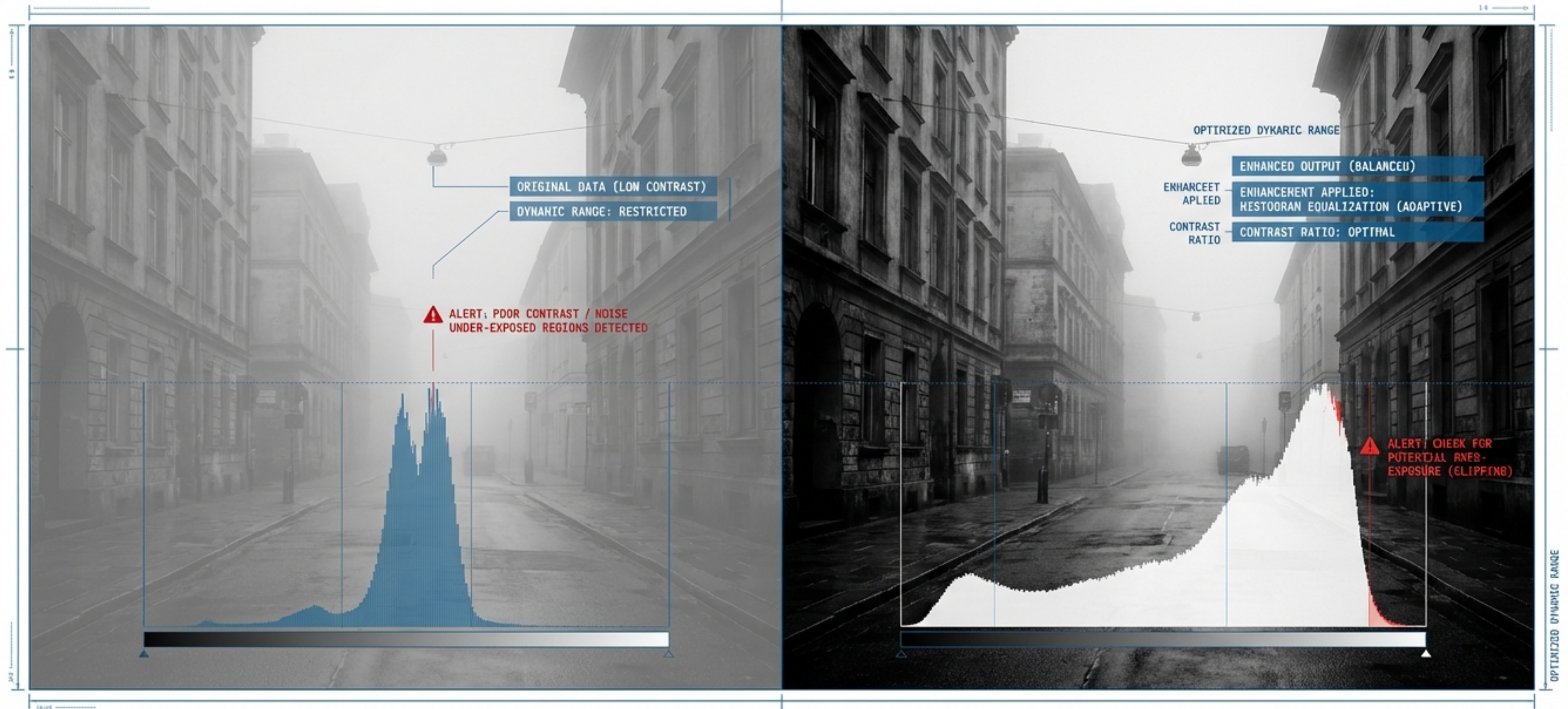


CHƯƠNG 2: NÂNG CẤP VÀ KHÔI PHỤC ẢNH

Tiết 2: Biểu đồ Histogram & Các kỹ thuật Cân bằng nâng cao

Môn học: Thị giác máy tính (INF03111) Lớp: IT23M Giảng viên: Trần Thành Thắng



Mục tiêu Tiết học: Từ "Chẩn đoán" đến "Điều trị"



Hiểu bản chất thống kê: Nắm rõ cách Histogram biểu diễn tần suất xuất hiện của các mức **cường độ sáng (pixel)**.



Kỹ năng "Bắt bệnh": Nhìn biểu đồ → Đọc vị ngay tình trạng ảnh (Thiếu sáng, Dư sáng, Tương phản thấp).

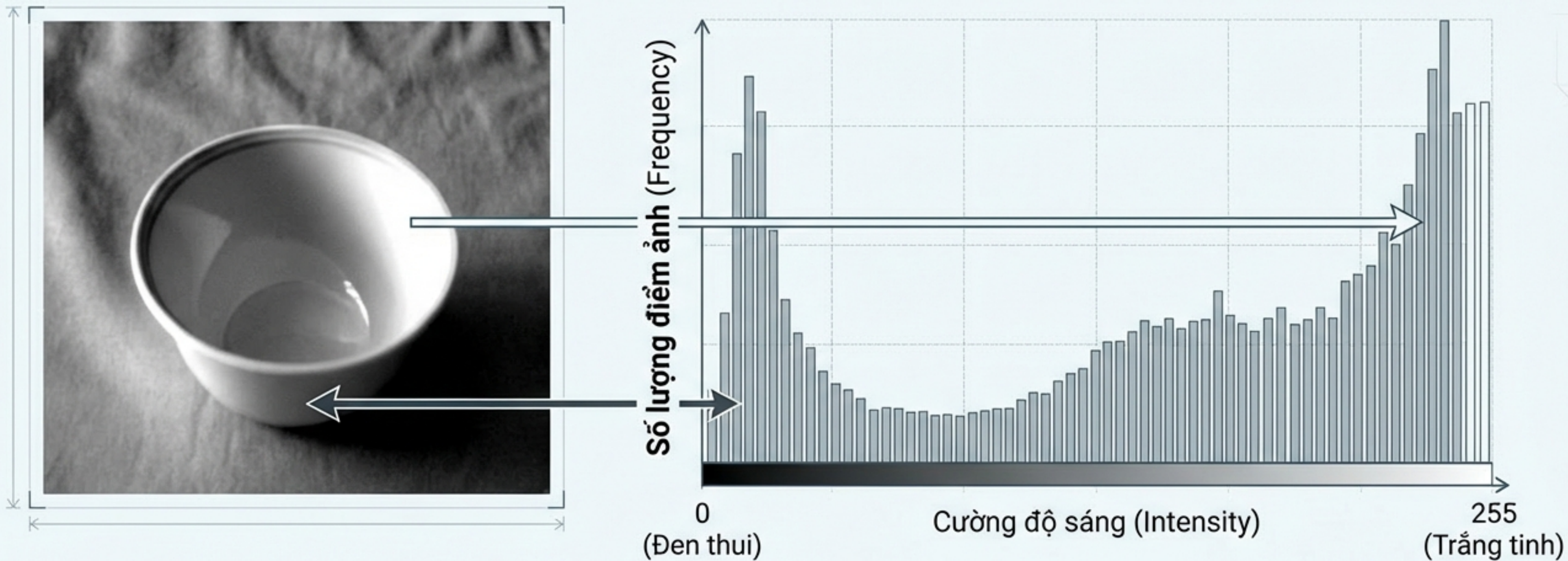


Phân tích rủi ro: Tại sao **Cân bằng toàn cục (Global HE)** mạnh nhưng dễ gây "**tác dụng phụ**"? ⚠



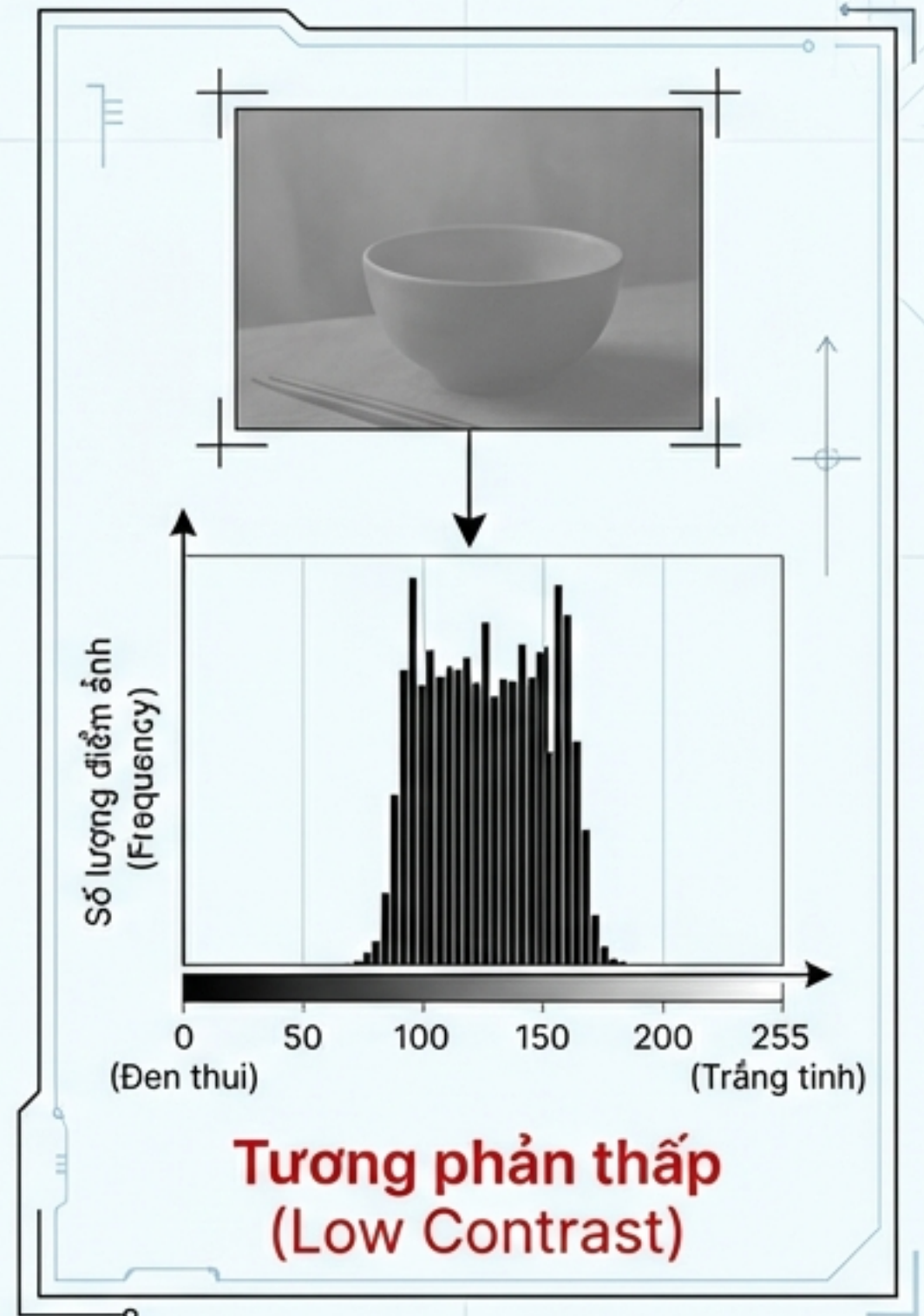
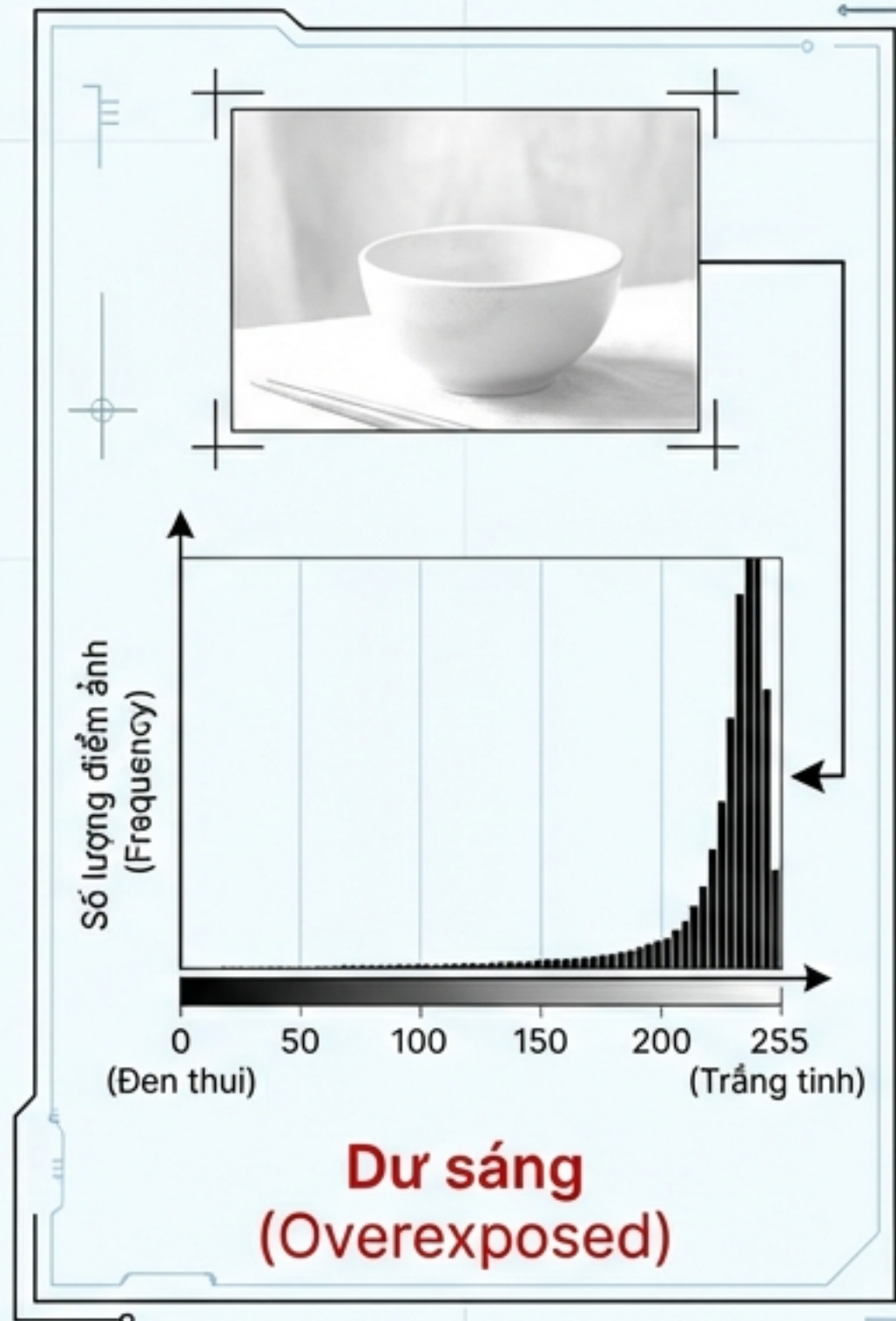
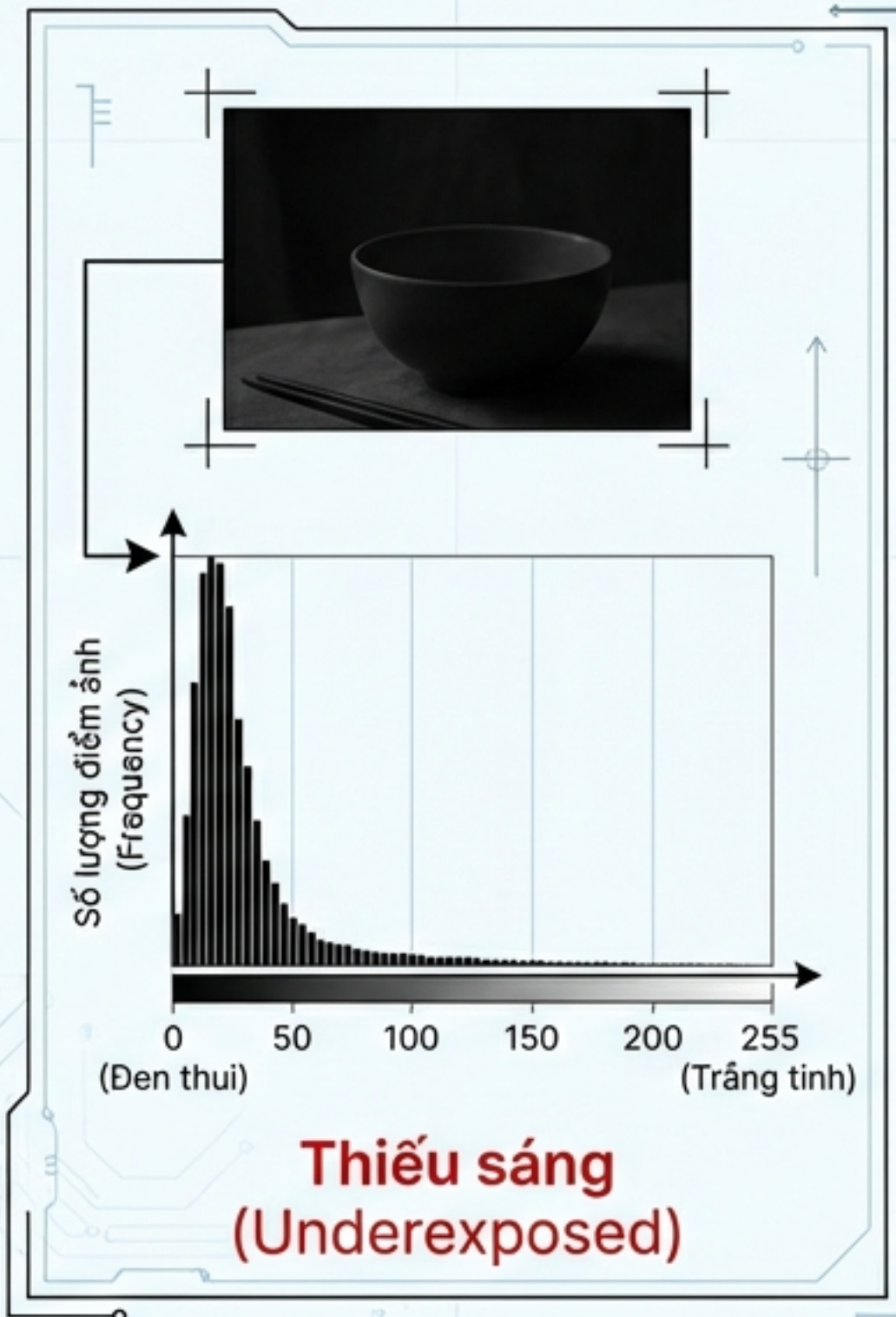
Làm chủ công nghệ: Áp dụng **CLAHE** để xử lý ảnh phức tạp (ngược sáng, sương mù).

Histogram (Lược đồ xám) là gì?

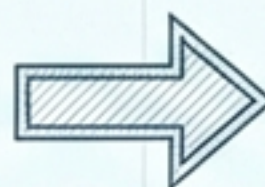
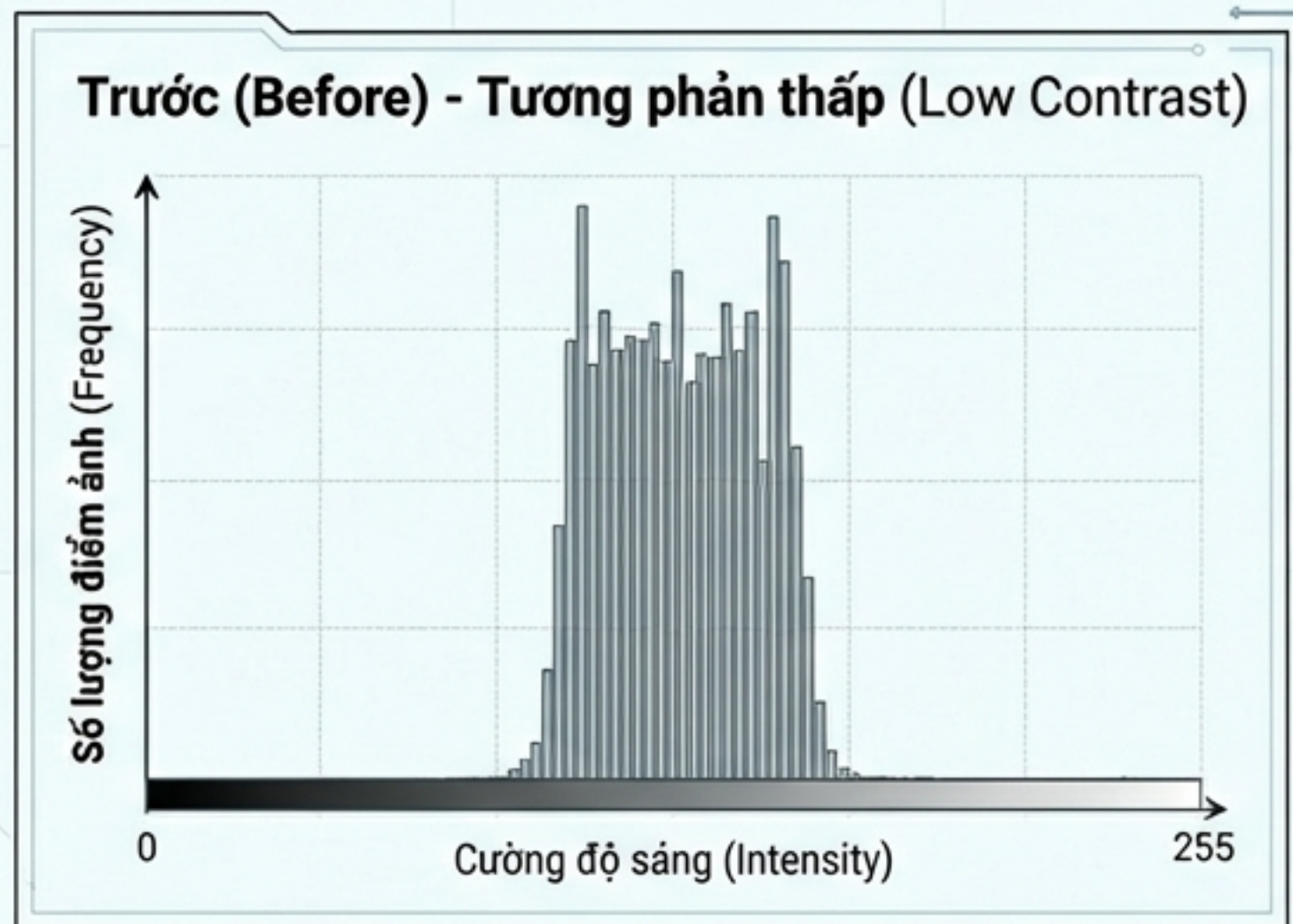


Máy tính không “nhìn” thấy hình ảnh như con người.
Nó dùng **Histogram** để “cảm nhận” sự phân bố ánh sáng tổng thể.

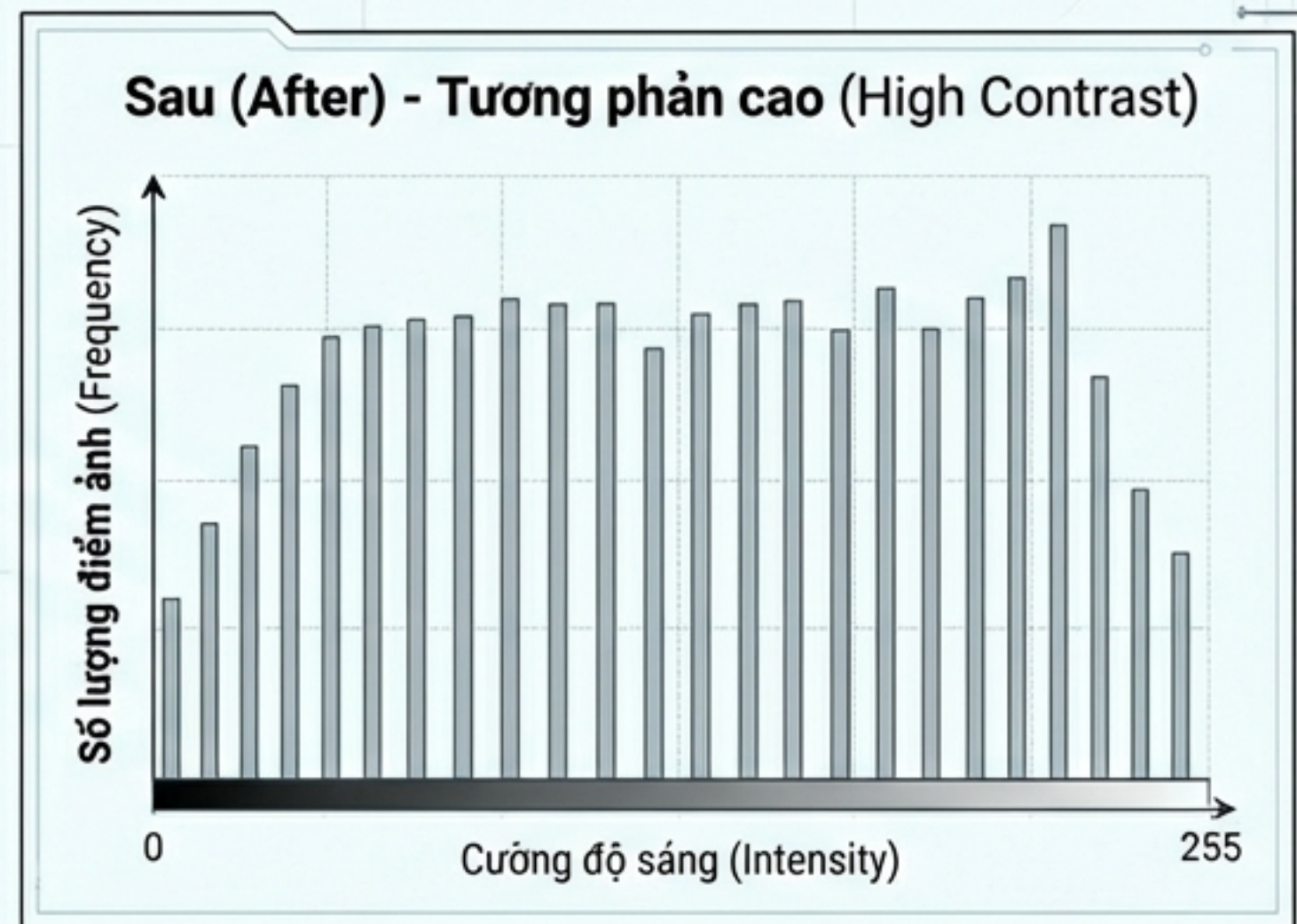
Kỹ năng "Bắt bệnh" qua biểu đồ



Phương pháp 1: Cân bằng Histogram toàn cục (Global HE)



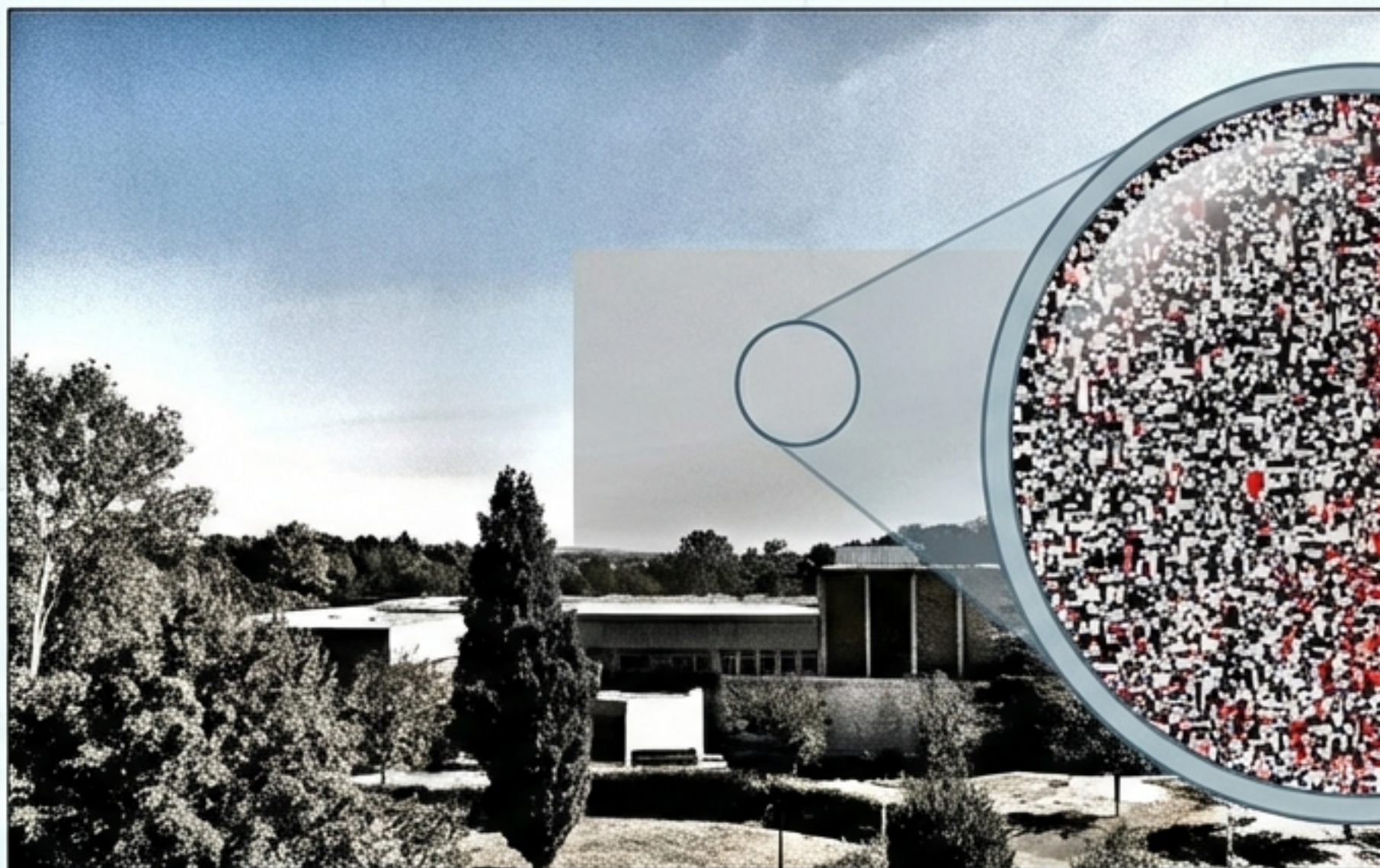
**Kéo giãn
(Stretch)**



```
cv2.equalizeHist(image)
```

Nguyên lý: "Kéo giãn" biểu đồ để phân bố lại các mức xám trên toàn bộ dải $[0, 255]$.
Tăng độ tương phản tổng thể ngay lập tức.

Tác dụng phụ của Global HE: Khi "Thuốc" quá mạnh

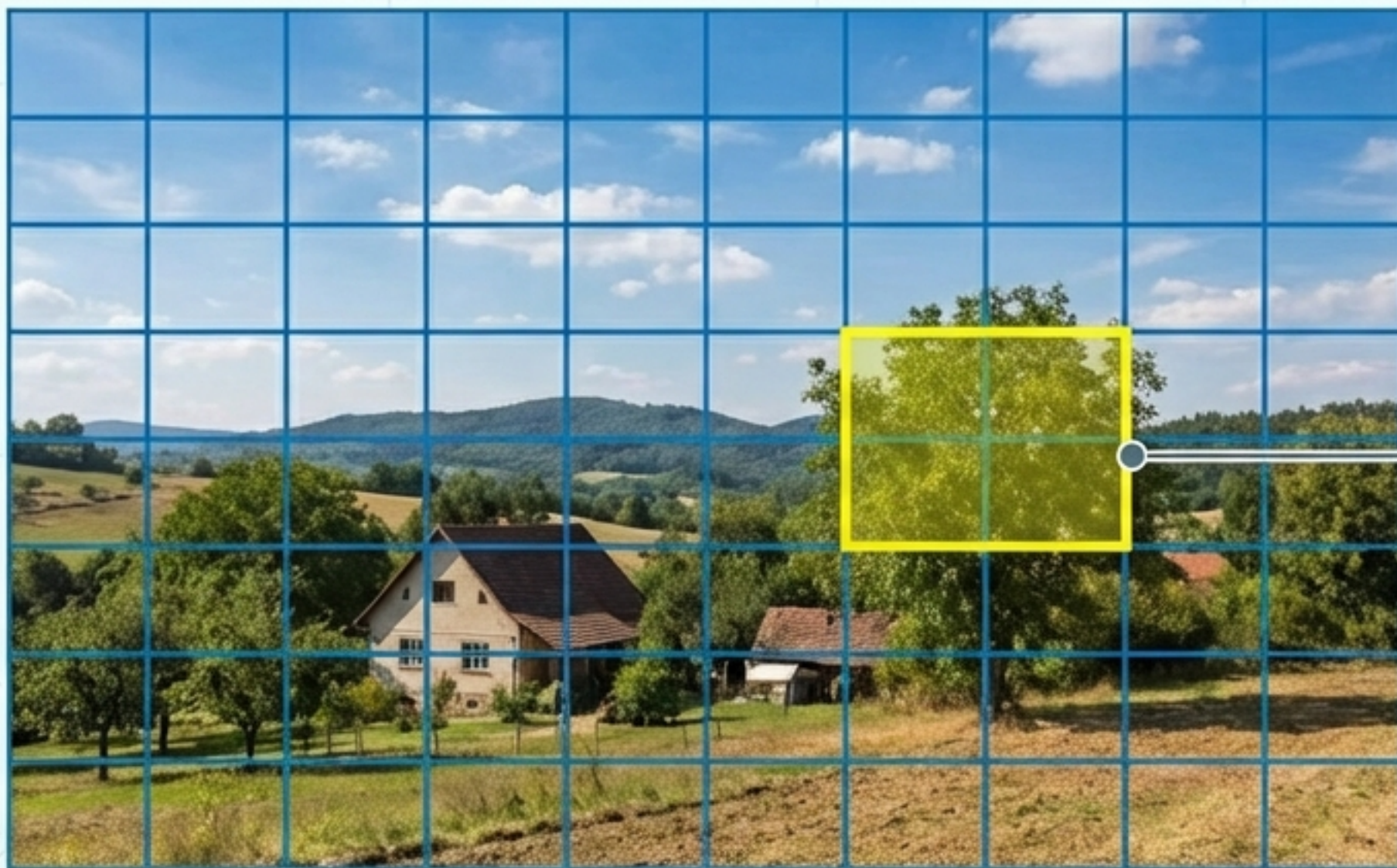


**Nhiều hạt (Noise)
bị phóng đại**

- **Vấn đề:** Thuật toán xử lý toàn bộ ảnh như một khối thống nhất.
- **Hậu quả:** Các vùng tối hoặc đồng nhất bị kéo giãn quá mức.
- **Kết luận:** Cần một phương pháp "phẫu thuật" chính xác hơn.

Giải pháp: CLAHE

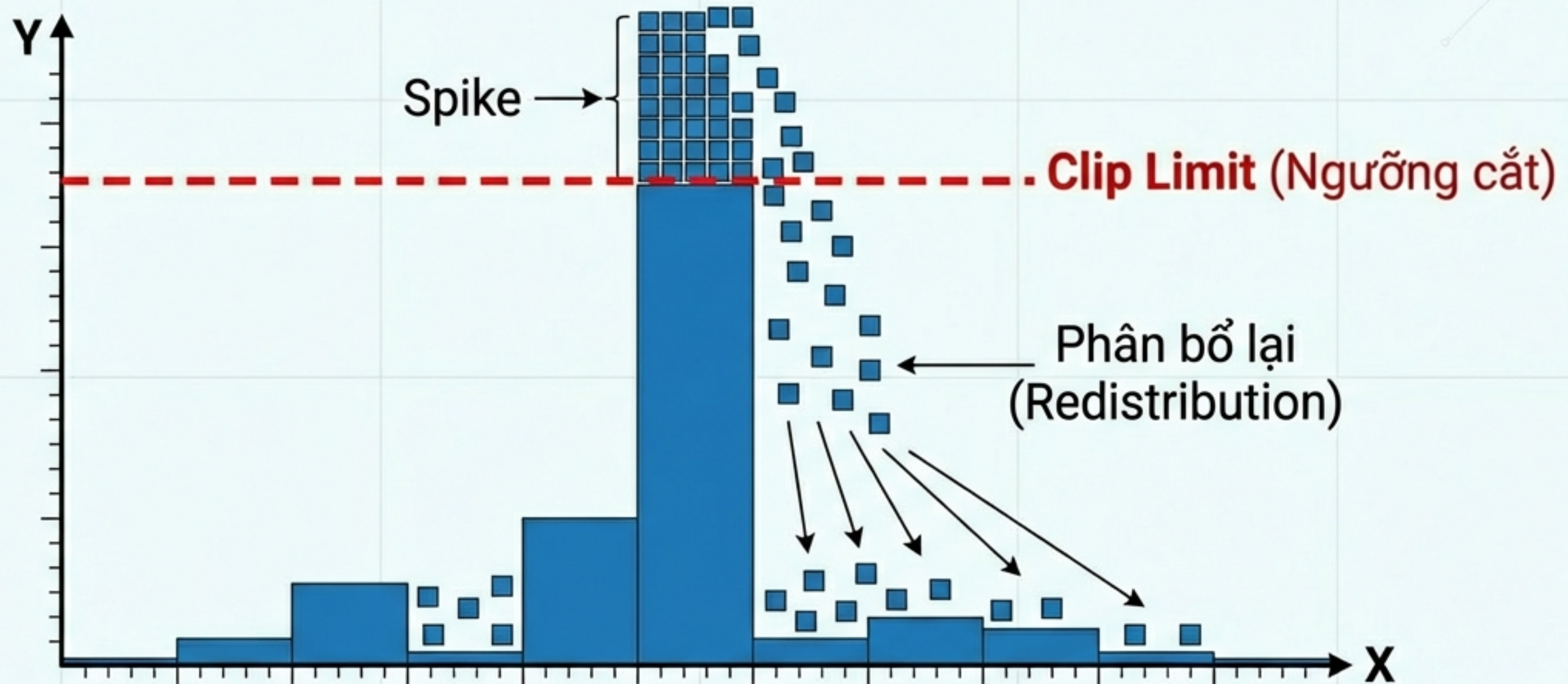
Contrast Limited Adaptive Histogram Equalization



Tính toán cục bộ (Adaptive)

- **Adaptive (Thích nghi):** Chia ảnh thành lưới (Grid), ví dụ 8x8.
- **Cơ chế:** Cân bằng histogram riêng cho từng ô lưới nhỏ thay vì toàn bộ ảnh.
- **Lợi ích:** Giữ lại chi tiết cục bộ, tránh làm hỏng các vùng ảnh vốn đã tốt.

Cơ chế chặn nhiễu: Clip Limit



- **Ngưỡng cắt:** Chặn đứng các cột pixel cao bất thường (nguồn gốc của nhiễu).
- **Phân bổ lại:** Phần bị cắt được rải đều cho các mức xám khác.

So sánh hiệu quả thực tế

Ảnh gốc (Original)



Chất lượng ảnh gốc với
tương phản cực đoan.

Global HE



→ **Nhiều hạt, Cháy sáng**

CLAHE



→ **Cân bằng, Giữ chi tiết**

CLAHE là lựa chọn tối ưu cho ảnh y tế, ảnh vệ tinh, và ảnh điều kiện ánh sáng phức tạp.

Kỹ thuật mở rộng: So khớp Histogram (Matching)

"Copy" phong cách ánh sáng



Ảnh gốc (Source)

+



Ảnh mẫu (Reference)

=



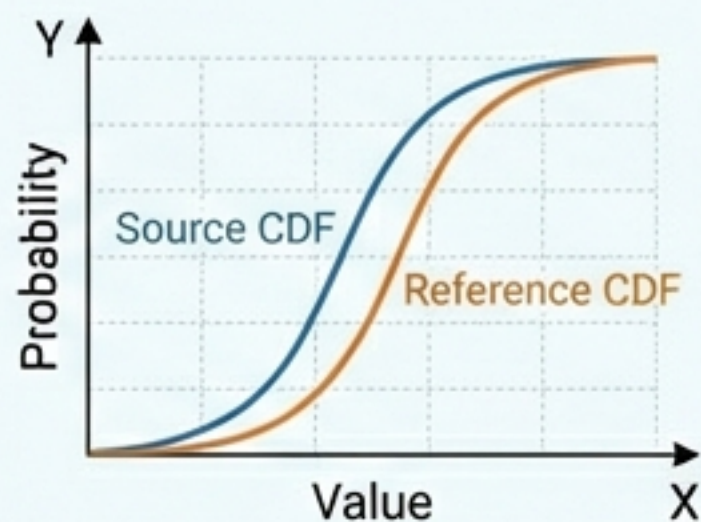
Kết quả (Result)

Ứng dụng (Application):

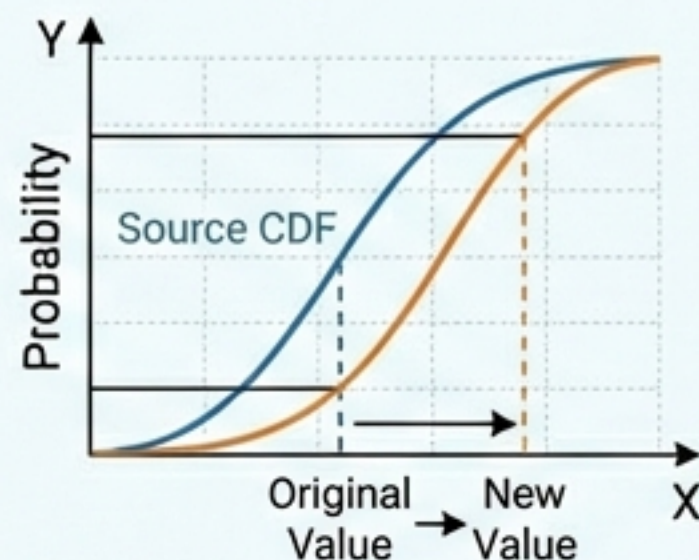
- Đồng bộ màu sắc camera, Color Grading (chỉnh màu nghệ thuật).

Toán học đằng sau sự “Bắt chước”

Tính CDF

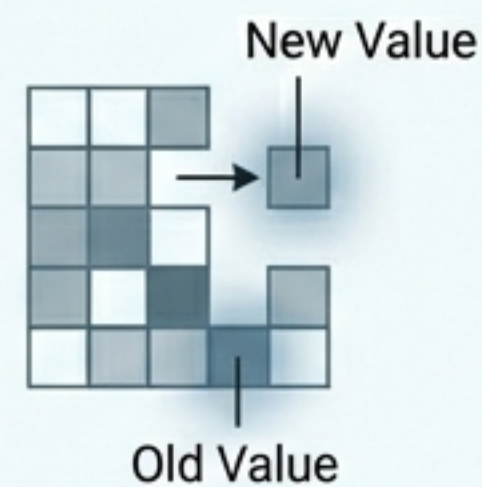


Mapping (Lập bản đồ)



Tìm cặp giá trị sao cho
 $CDF_{gốc} \approx CDF_{mẫu}$

Gán lại Pixel



Key Insight: Ép phân bố xác suất của ảnh này đi theo dáng điệu của ảnh kia.

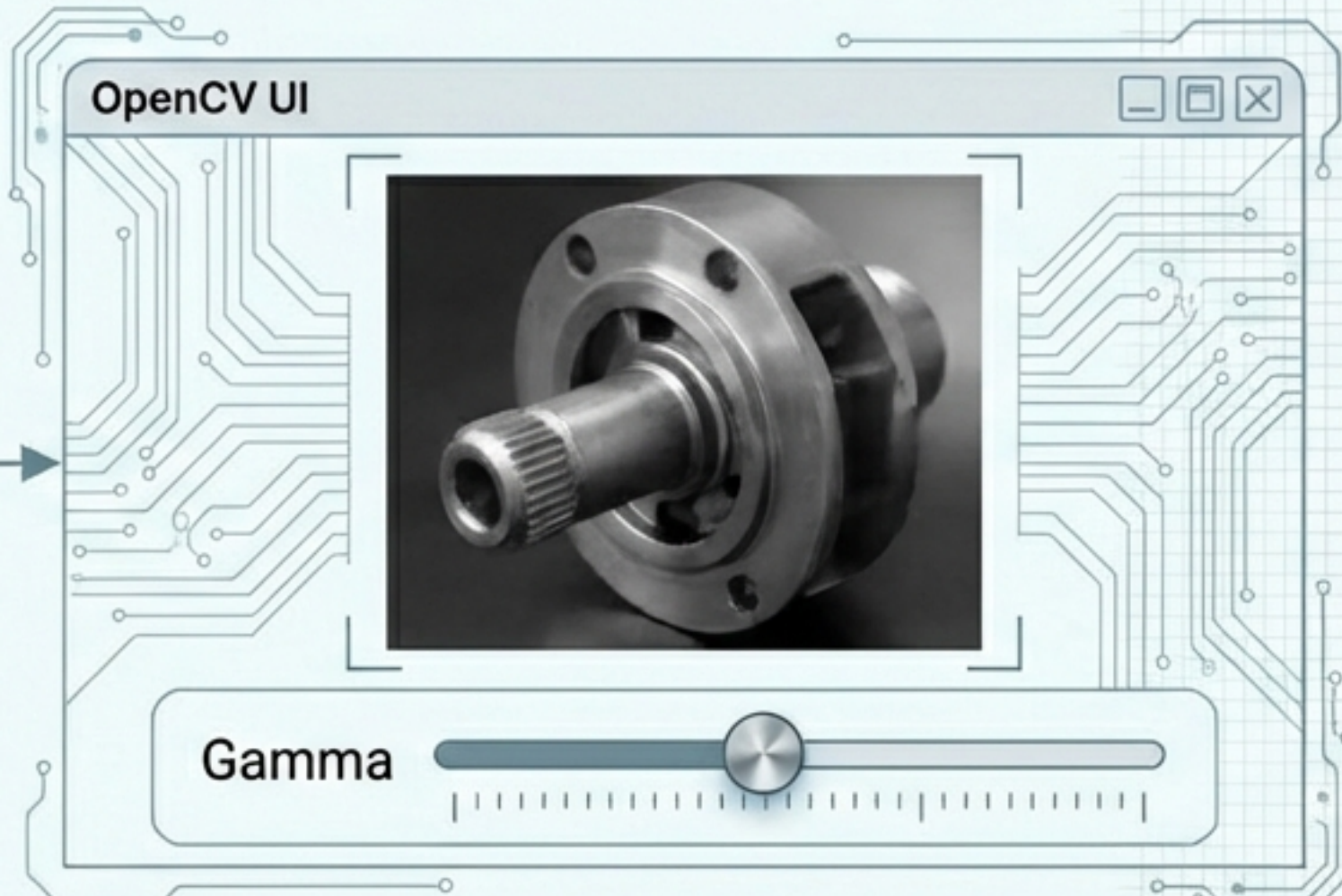
Tổng kết: Lựa chọn "Vũ khí" xử lý ảnh

Kỹ thuật	Khi nào dùng?	Ưu/Nhược điểm
Global HE (`cv2.equalizeHist`)	Ảnh tối đều, cần xử lý nhanh.	+ Đơn giản. - Mất chi tiết, nhiều.
CLAHE (`cv2.createCLAHE`)	Ảnh phức tạp (X-quang, sương mù, ngược sáng).	+ Giữ chi tiết tốt. - Cần chỉnh tham số.
Histogram Matching	Cần đồng bộ màu sắc, filter nghệ thuật.	+ Linh hoạt. - Tính toán phức tạp.

Chuẩn bị Lab 2.1: Tối ưu hóa với Look-up Table (LUT)

```
# Tạo bảng LUT (Look-up Table)
invGamma = 1.0 / gamma
table = np.array([(i / 255.0) ** invGamma) * 255
                 for i in range(0, 256)]).astype("uint8")

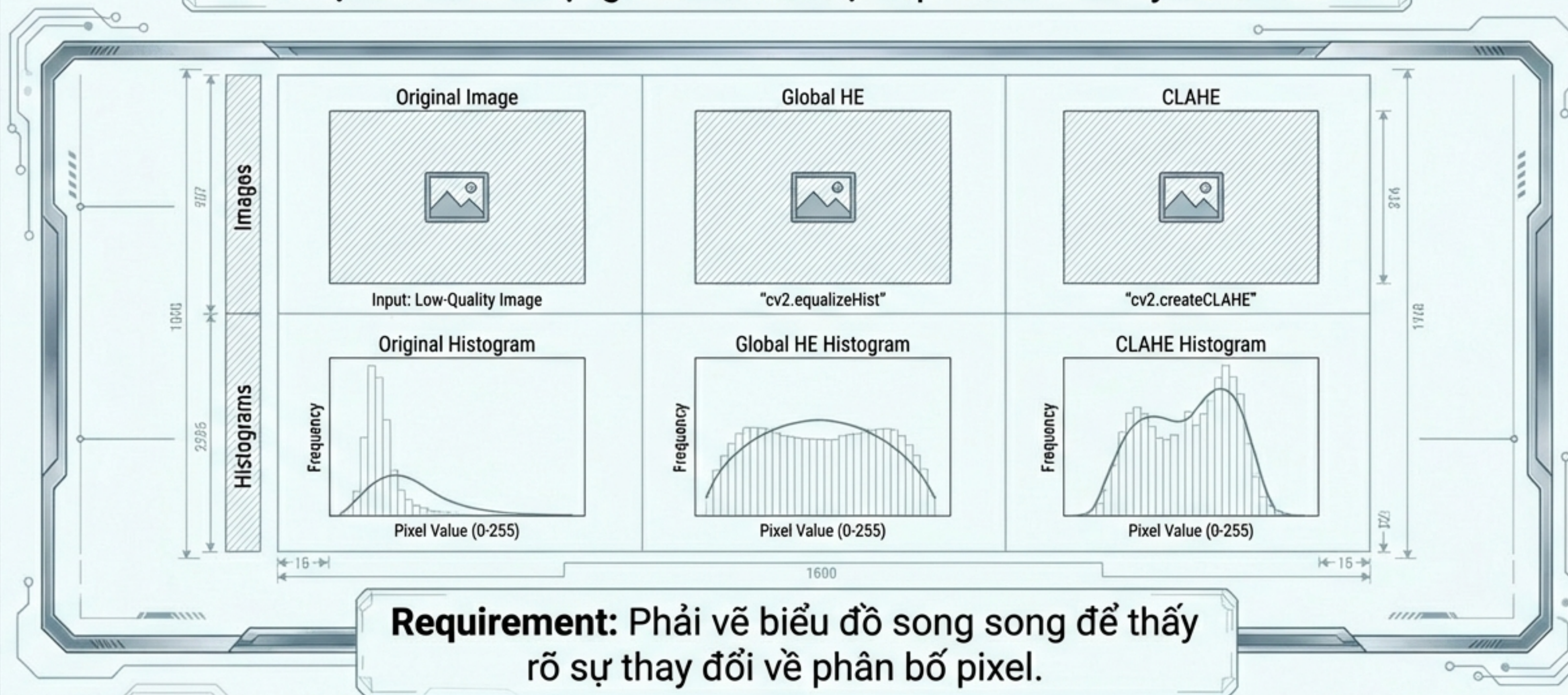
# Áp dụng siêu tốc
result = cv2.LUT(image, table)
```



Thay vì tính toán lại từng pixel (chậm),
hãy tính sẵn kết quả vào bảng (LUT)
và tra cứu (nhANH).

Nhiệm vụ thực hành: So sánh đối đầu

Load một ảnh chất lượng kém và cài đặt 3 phiên bản xử lý để so sánh.



Clinical Precision meets Engineering Blueprint

Thử thách: Kiểm tra thường xuyên **A1**

THÔNG BÁO KIỂM TRA



Thời gian: 15 - 20 phút (Trong buổi thực hành Tiết 3).



Hình thức: Code trực tiếp trên máy.



Chủ đề: Gamma Correction + Histogram.



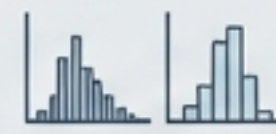
Topic Hint:



Cho ảnh đầu
vào bị tối



Dùng Gamma
Correction làm sáng



Vẽ Histogram
trước/sau



Pro Tip: Ôn kỹ cách dùng **matplotlib** để vẽ biểu đồ và **'cv2.LUT'** để code chạy nhanh!

Requirement: Phải vẽ biểu đồ song song để thấy rõ sự thay đổi về phân bố pixel.